



计算机科学与探索

Journal of Frontiers of Computer Science and Technology

ISSN 1673-9418, CN 11-5602/TP

《计算机科学与探索》网络首发论文

题目：融合全局图拓扑与多尺度掩码卷积的漏洞检测方法
作者：黄安博，曲海成，姜庆玲
网络首发日期：2025-09-22
引用格式：黄安博，曲海成，姜庆玲. 融合全局图拓扑与多尺度掩码卷积的漏洞检测方法[J/OL]. 计算机科学与探索.
<https://link.cnki.net/urlid/11.5602.tp.20250919.1235.004>



网络首发：在编辑部工作流程中，稿件从录用到出版要经历录用定稿、排版定稿、整期汇编定稿等阶段。录用定稿指内容已经确定，且通过同行评议、主编终审同意刊用的稿件。排版定稿指录用定稿按照期刊特定版式（包括网络呈现版式）排版后的稿件，可暂不确定出版年、卷、期和页码。整期汇编定稿指出版年、卷、期、页码均已确定的印刷或数字出版的整期汇编稿件。录用定稿网络首发稿件内容必须符合《出版管理条例》和《期刊出版管理规定》的有关规定；学术研究成果具有创新性、科学性和先进性，符合编辑部对刊文的录用要求，不存在学术不端行为及其他侵权行为；稿件内容应基本符合国家有关书刊编辑、出版的技术标准，正确使用和统一规范语言文字、符号、数字、外文字母、法定计量单位及地图标注等。为确保录用定稿网络首发的严肃性，录用定稿一经发布，不得修改论文题目、作者、机构名称和学术内容，只可基于编辑规范进行少量文字的修改。

出版确认：纸质期刊编辑部通过与《中国学术期刊（光盘版）》电子杂志社有限公司签约，在《中国学术期刊（网络版）》出版传播平台上创办与纸质期刊内容一致的网络版，以单篇或整期出版形式，在印刷出版之前刊发论文的录用定稿、排版定稿、整期汇编定稿。因为《中国学术期刊（网络版）》是国家新闻出版广电总局批准的网络连续型出版物（ISSN 2096-4188，CN 11-6037/Z），所以签约期刊的网络版上网络首发论文视为正式出版。

融合全局图拓扑与多尺度掩码卷积的漏洞检测方法

黄安博¹, 曲海成¹⁺, 姜庆玲²

1. 辽宁工程技术大学 软件学院, 辽宁 葫芦岛 125000

2. 铁岭师范高等专科学校, 理学院, 辽宁 铁岭 112000

+ 通信作者 E-mail: quhaicheng@lntu.edu.cn

摘要：基于序列的深度学习方法在建模源代码的结构特征方面存在不足，而图神经网络（Graph Neural Networks, GNN）虽然可以通过聚合邻居节点信息丰富当前节点表征，但无法有效获取图的全局特征信息，且难以捕获图节点间长距离依赖。为克服上述问题，本文提出了一种融合全局图拓扑与多尺度掩码卷积的门控漏洞检测方法（GTMC-VD）。在该方法中，利用开源工具（Joern）将源代码转换为代码属性图（Code Property Graph, CPG），采用词嵌入模型（Word2vec）对图中节点进行嵌入以获得图中节点的初始表示，设计并实现了图全局拓扑编码器，该编码器利用图卷积网络（Graph Convolutional Networks, GCN）的输出作为节点重要性评分，利用该评分对图结构进行简化，并对邻接矩阵和节点特征进行更新，通过层次化的策略实现逐层优化并采用不同池化层获取多尺度的拓扑信息，最终聚合以获取图的全局特征；之后采用两个不同尺度的卷积核捕捉节点之间的依赖关系，同时，针对变长图数据引入掩码机制，避免因填充节点带来的噪声干扰，实现了多尺度掩码卷积模块。最终，引入门控机制，自适应融合两个模块的输出结果，并得到模型最终检测结果。在两个公开数据集上的大量实验表明，所提方法有效解决了上述两个问题，并在准确率（Accuracy）、精确率（Precision）、召回率（Recall）和 F_1 分数（ F_1 Score）指标上相比于基准模型（Devign）分别提高了 6.69%、4.43%、13.63% 和 8.17%。总之，GTMC-VD 有效获取了图的全局特征，且缓解了基于 GCN 模型无法捕捉长距离依赖的问题，为漏洞检测任务提供了一种更为鲁棒且高效的解决方案。

关键词：漏洞检测；图神经网络；图拓扑；多尺度卷积；掩码机制

文献标志码：A **中图分类号：**TP183

A Gated Vulnerability Detection Method Integrating Global Graph Topology and Multi-Scale Masked Convolution

HUANG Anbo¹, QU Haicheng¹⁺, JIANG Qingling²

1. College of Software, Liaoning Technical University, Huludao, Liaoning 125000, China

2. College of Science, Tieling Normal College, Tieling, Liaoning 112000, China

Abstract: Sequence-based deep learning methods exhibit limitations in modeling the structural characteristics of source code. Although graph neural networks (GNNs) can aggregate neighborhood information to enrich node representations, they struggle to effectively capture global graph representations and long-range dependencies among nodes. To address these challenges, this paper proposes a novel approach named GTMC-VD (Global graph Topology and Multi-Scale masked Convolution). In this method, the open-source tool (Joern) is used to convert the source code into the Code Property Graph (CPG), and the word embedding model (Word2vec) is used to embed the nodes in the graph to obtain the initial representation of the nodes. A global topology encoder is designed and implemented, which uses the output of the Graph Convolutional Networks (GCN) as the node importance score, and uses the score to simplify the graph structure and update the adjacency matrix and node features. Through a hierarchical strategy, the topology information is optimized layer by layer, and the multi-scale topology information is obtained by using different pooling layers. Finally, the topology information is aggregated to obtain the global features of the graph. After that, two different scale convolutional kernels are used to capture the dependencies between nodes. At the same time, a masking mechanism is introduced for variable-length graph data to avoid noise interference caused by padding nodes, and a multi-scale masked convolution module is realized. Finally, a gating mechanism is introduced to adaptively fuse the output results of the two modules, and the final detection results of the model are obtained. A large number of experiments on two public datasets show that the proposed method effectively solves the above two problems, and improves the accuracy (Accuracy), precision (Precision), recall (Recall) and F_1 score (F_1 Score) indicators compared to the baseline model (Devign) by 6.69%, 4.43%, 13.63% and 8.17% respectively. In short, GTMC-VD effectively obtains the global features of the graph, and alleviates the problem that the GCN model cannot capture long-range dependencies, providing a more robust and efficient solution for vulnerability detection tasks.

基金项目：国家自然科学基金面上项目（42271409）；辽宁省教育厅基本科研项目（No. JYTMS20230804）。

This work was supported by the National Natural Science Foundation of China (42271409), and the Scientific Research Foundation of the Higher Education Institutions of Liaoning Province, China (Grant No. JYTMS20230804)

Convolution Network for Vulnerability Detection). In this method, source code is first transformed into a Code Property Graph (CPG) using the Joern tool. Subsequently, the Word2Vec algorithm is applied to embed nodes within the graph, generating initial node representations. We design and implement the Global Graph Topology Encoder that leverages GCN (Graph Convolutional Networks) outputs as node importance scores. The graph structure is then simplified via the score, updating the adjacency matrix and node features accordingly. A hierarchical pooling strategy is employed to get multi-scale topological information and aggregate it to obtain comprehensive global graph representation. Within the Multi-Scale masked Convolution module, two convolutional kernels of different scales are used to capture relationships between both distant and neighboring nodes. Additionally, a masking mechanism is introduced to handle variable-length graph data, mitigating noise caused by padding nodes. Finally, a gating mechanism adaptively fuses the outputs of the two component to produce the final vulnerability detection result. Extensive experiments on two public datasets demonstrate that the proposed method effectively addresses the aforementioned two issues and achieves improvements of 6.69%, 4.43%, 13.63%, and 8.17% in Accuracy, Precision, Recall, and F1 Score, respectively, compared to the baseline model (Devign). In summary, GTMC-VD effectively captures global graph features while mitigating the limitation of GCN-based models in capturing long-range dependencies, providing a more robust and effective solution for vulnerability detection tasks.

Key words: Vulnerability Detection; Graph Neural Networks; Graph Tology; Multi-Scale Convolution; Mask mechanism

随着开源生态系统的迅猛发展, GitHub 等平台的开发者预计将超过 1 亿(截至 2025 年), 且涌现出大量开源项目。据 Synopsys 旗下 Black Duck 审计服务团队在 2024 年对 1067 个代码仓库的分析显示, 其中高达 96% 与开源项目相关^[1]。分析结果表明, 接受风险评估的代码仓库中有 84% 存在至少一个已知的开源漏洞, 更严重的是, 有 74% 的代码仓库包含高风险漏洞, 较 2022 年显著增加。这些开源漏洞加剧了单点漏洞引发级联安全风险的可能性, 对开源生态构成了严峻威胁。因此, 迫切需要大规模、智能化的软件漏洞检测方法, 以提升软件安全保障能力。

根据检测过程所采用的技术, 源代码漏洞检测方法主要可分为基于规则的方法与基于学习的方法。

基于规则的源代码漏洞检测方法(例如部分开源或商用的漏洞检测工具)依赖于专家预定义的规则。由于专家主观性强, 这类规则往往存在不完整性与局限性, 易导致误报与漏报^[2]。此外, 随着安全漏洞数量不断增长, 这类方法需投入大量人工进行规则维护, 进而提高了人力成本^[3]。

相比之下, 基于学习的漏洞检测方法可以从历史数据中自动学习漏洞模式, 避免对专家知识的依赖^[3]。深度学习在计算机视觉(Computer Vision, CV)与自然语言处理(Natural Language Processing, NLP)领域取得的显著成就为漏洞检测提供了重要启示。例如, 由于编程语言(Program Language, PL)与自然语言(Natural Language, NL)均遵循一定的语法规则组织

代码或语句, 并体现出上下文依赖性, 部分研究借鉴 NLP^[4-6]方法用于漏洞检测。然而, NLP 方法往往无法充分建模源代码中蕴含的结构信息, 导致检测性能受限。

为克服上述问题, 一些研究者^{[7][8]}引入程序分析手段, 开始从源代码中提取程序语义并构建图结构, 通过图分析进行漏洞检测。这类方法日益受到重视。与规则排列的图像像素(2D/3D 网格)或自然语言中顺序排列的词、字符(1D 序列)不同, 图数据属于非欧几里得结构, 由节点(实体)与边(关系)组成, 具备不规则性与复杂的关系模式。这些特性使图能够更加准确地表示源代码, 从而提升检测效果。然而, 相较于 NLP 方法, 图方法在可扩展性方面存在不足。此外, GNN^[9]中的信息传递机制也引发两个关键问题: 一方面, 浅层 GNN 难以捕捉节点之间的远距离依赖关系^[10]; 另一方面, 现有 GNN 方法往往无法直接获取图的全局特征, 影响了检测性能。

为应对上述挑战, 本文提出了一种融合全局图拓扑与多尺度掩码卷积的漏洞检测方法(Global graph Topology and Multi-Scale masked Convolution Network for Vulnerability Detection, GTMC-VD)。在该方法中, 利用 Joern 将源代码转换为 CPG, 采用 Word2vec^[11]对图中节点进行嵌入以获得图中节点的初始表示, 设计并实现了图全局拓扑编码器, 该编码器利用 GCN 的输出作为节点重要性评分, 利用该评分对图结构进行简化, 并对邻接矩阵和节点特征进行更新, 采用层次

化的策略实现逐层优化并利用不同池化层获取多尺度的拓扑信息,最终聚合以获取图的全局特征;之后采用两个不同尺度的卷积核捕捉节点之间的关系,同时,针对变长图数据引入掩码机制,避免因填充节点带来的噪声干扰,实现了多尺度掩码卷积模块。最终,引入门控机制,自适应融合两个模块的输出,并得到模型最终检测结果。

1 相关工作

近年来,深度学习技术在漏洞检测领域取得了显著进展。根据模型架构的不同,现有方法大致可分为序列程序表示的深度学习模型和抽象语法树或图结构程序表示的深度学习模型。

1.1 序列程序表示的深度学习模型

此类方法通常将源代码视为序列数据,并利用多种神经网络结构进行特征学习与漏洞检测。因此,首先需要将源代码转换为适当的序列表示,以有效建模代码中蕴含的丰富语法与语义信息。

一些研究者^[12-14]提出将源代码或代码片段表示为词元序列。这类方法通常采用词袋模型(N-gram)^[15]模型或 Word2Vec 将代码词元转换为向量表示,并输入顺序神经网络进行处理。例如,Russell^[12]等人使用 Word2Vec 对代码进行嵌入,结合卷积神经网络(Convolutional Neural Network,CNN)^[16]和循环神经网络(Recurrent Neural Network,RNN)^[17]架构分别提取局部与全局特征。类似地,Zhen Li 等人提出了 Vul-DecPecker^[18]和 SySeVR^[19],通过不同方式将源代码划分为包含不同层次语法与语义信息的片段,并采用门控循环单元(Gated Recurrent Unit,GRU)^[20]和长短期记忆网络(Long Short-Term Memory,LSTM)^[21]作为漏洞特征提取的主干网络。

为了更好地捕捉代码中的上下文信息,许多研究^{[22][23]}采用了双向架构,例如,双向门控循环单元(Bidirectional Gated Recurrent Unit,BiGRU)或双向长短期记忆网络(Bidirectional Long Short-Term Memory,BiLSTM)。与单向网络相比,双向网络同时集成了前向层与后向层,能在每个时间步产生基于双向隐藏状态的输出,从而有效建模代码中前后文的依赖关系。

近年来,随着深度学习的发展,Transformer^[24]架

构因其在建模长距离依赖方面的卓越能力而受到广泛关注。与 CNN 和 RNN 相比,Transformer 可以直接计算任意两个位置标记或语句之间的依赖关系,从而更好地捕捉全局和长距离的依赖关系。W.Tao^[25]等人使用 Transformer 进行漏洞检测方面取得了显著的成果。在此基础上,大语言模型(Large Language Model,LLMs)也逐渐被引入漏洞检测,并成为新的研究热点。现有研究表明,漏洞检测任务多采用 encoder-only 架构的小规模模型^[4]。在方法上,微调仍占主导地位,例如 VulBERTa^[26]在 C/C++ 上的领域化预训练显著提升了性能;同时,对比学习与因果推断方法(如 CausalVul^[27])被用于增强模型鲁棒性。此外,提示工程(如注入 CWE 信息或数据流提示)与检索增强(如基于 CVE 知识库的 RAG)^{[28][29]}也被用于提升检测的准确性与可解释性。

尽管上述方法已取得一定成效,但它们仍存在固有局限性:PL 所包含的逻辑结构远比简单的词元序列更加严谨。例如,循环体内外语句在序列上相邻,但在语义上却可能毫无关联。综上所述,尽管基于序列的深度学习模型能够有效学习顺序语义特征,但仍难以捕捉源代码具有的结构语义与语法信息。

1.2 抽象语法树或图结构程序表示的深度学习模型

抽象语法树或图结构程序表示的深度学习模型通常首先将源代码转换为结构化表示(如语法树或程序图),随后利用相应的神经网络进行特征提取与检测。这类方法与基于序列的模型根本区别在于,其明确保留并利用了源代码中所蕴含的丰富的结构信息。

基于语法树的代码表示方法通过将源代码解析为抽象语法树(Abstract Syntax Tree,AST)并建模其结构,有效保留了语法结构信息,广泛应用于各类语义相关任务中。然而,已有研究表明^[30]:AST 通常包含大量与漏洞识别无关的冗余节点,限制了其检测效果。具体而言,虽然 AST 能全面捕捉代码的语法结构,但对程序中的控制逻辑、数据依赖等对漏洞检测至关重要的信息表达不足。因此,仅依赖原始 AST 作为中间表示常常导致检测性能不佳。为克服这一问题,近期研究趋势倾向于通过引入控制流图(Control Flow Graph,CFG)、数据流图(Data Flow Graph,DFD)等程

序分析信息对 AST 进行增强,从而提升表示的准确性与鲁棒性。在此背景下,基于图结构的程序表示方法受到了越来越多的关注。例如,Zhou 等人^[31]提出采用门控图神经网络 (Gated Graph Sequence Neural Networks,GGNN)^[32],结合门控单元以加强复合代码图中结构信息与顺序信息的联合学习。

GCN 作为图学习的基础模型,常作为漏洞检测任务中的基线方法^{[33][34]}。为进一步提升检测性能,部分研究工作^[35]引入了关系图卷积网络 (Relational Graph Convolutional Networks,R-GCN),通过关系感知机制优化异构图中的表示学习。此外,Li^[36]等人提出了特征注意力图卷积网络,引入注意力机制以加强关键节点特征的提取。近年来,图注意力网络(Graph Attention Networks,GAT)^[37]因其出色的选择性特征聚合能力,在漏洞检测任务中取得了显著成果。Li 等人与 Ghaffarian^[38]等人的实验结果表明,基于 GAT 的模型能够有效学习代码图中深层的依赖关系,并在多项对比研究中达到了当前最优性能。

总体而言,这些研究为图神经网络在漏洞检测领域的进一步发展提供了重要参考。然而,这些模型往往专注于丰富节点表征进行图表示学习并未获取图的全局特征,且浅层 GCN 的表达能力有限,无法获取节点间的长距离依赖,导致检测性能受限。而 GAT 往往参数量过大,运算成本较高。

2 方法

2.1 问题定义

本文将函数级别的源代码漏洞检测任务视为一个图的二分类问题,其中分类结果为 0 表示图对应的源代码不包含漏洞,结果为 1 表示存在漏洞。将数据集表示为 $\{(c_i, y_i) \mid c_i \in C, y_i \in Y\}_{i=1}^N$, 其中 C 表示源代码集合, $Y = \{0, 1\}$ 表示对应的标签集合, N 表示数据集中代码数量。随后,源代码 C 被 Joern 转换为对应的 CPG 图 G 。本文的目标是通过最小化损失函数,学习一个映射函数 $f: G \Rightarrow Y$, 见公式 (1)。

$$\min \sum L(f(G_i, y_i \mid c_i)) \quad (1)$$

其中 L 表示交叉熵损失函数。

2.2 框架

GTMC-VD 的模型框架如图 1 所示。该模型由四个关键组成部分构成:(1) 数据集处理:负责将数据集中原始源代码转换为模型可接受的输入;(2) 图全局拓扑编码器:显式地对图拓扑结构进行处理,利用 GCN 提取节点重要性评分并简化图结构,采用不同的池化方式提取多尺度图信息,并采用多层设计逐层优化,从而提取图的全局特征;(3) 多尺度掩码卷积模块:采用两个不同大小的卷积核捕捉远近距离节点之间依赖,同时,针对变长图数据引入掩码机制,避免因填充节点带来的噪声;(4) 读出模块:使用门控机制动态融合两个通道的输出并生成最终的检测结果。

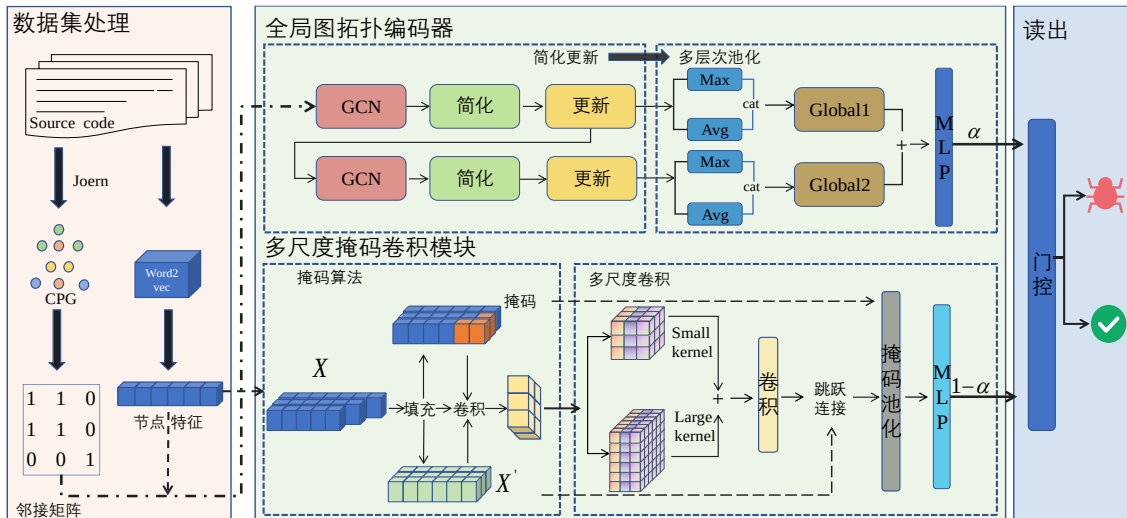


图 1 融合全局图拓扑与多尺度掩码卷积的漏洞检测方法模型架构图

Fig.1 GTMC-VD Model Architecture Diagram

2.2.1 数据集处理

本节介绍数据集处理, 通过 Joern 工具将源代码转换为适用于 GTMC-VD 处理的图结构表示, 本文采用 CPG 对源代码的结构和语义信息进行全面建模, 其中, 代码实体(如类、函数等)被表示为图中的节点, 程序语义(如数据流、控制流)通过边进行刻画。在将源代码转换为 CPG 之后, 利用 Word2vec 对图中节点的内容进行嵌入, 生成图的节点特征矩阵 $X \in R^{n \times d}$, 其中 n 表示图中的节点数量, d 表示节点特征维度, 此外获取图的邻接矩阵 $A \in R^{n \times n}$, 最后形成图数据 $G = (A, X)$ 。

2.2.2 图全局拓扑编码器

图全局拓扑编码器基于注意力分数驱动, 其中注意力分数来源于 GCN 输出。利用注意力分数对图进行简化, 将简化后的节点特征以及邻接矩阵输入至下一层中, 实现逐层优化, 并通过不同的池化操作保留多级特征信息, 最后融合各层特征信息, 从而获得图的全局特征。该设计不仅有效保留了关键的拓扑结构, 同时保持了网络各层之间的特征多样性, 能够提取完备的图的全局特征。

首先将图 G 输入到 GCN 中, 第 $l+1$ 层的输出见公式 (2)。

$$H^{(l+1)} = \alpha(\hat{D}^{-\frac{1}{2}} \hat{A} \hat{D}^{-\frac{1}{2}} H^{(l)} W^{(l)}) \quad (2)$$

其中, $A \in R^{n \times n}$ 表示图的邻接矩阵, $X \in R^{n \times d}$ 表示图的节点特征矩阵, $D \in R^{n \times n}$ 为邻接矩阵 A 的度矩阵, 用于归一化处理, $H^{(l)}$ 表示第 l 层 GCN 的输出, $W^{(l)}$ 为该层的权重矩阵。GCN 的输出记为 $H^{(l+1)} \in R^{1 \times n}$, 将 GCN 的输出维度设置为 1, 通过第一层 GCN 后, 此时 $H^{(l+1)}$ 即为第一层 GCN 的输出, 随后利用激活函数获取节点重要性评分 S , 见公式 (3)。

$$S = \text{Tanh}(H^{(l+1)}) \quad (3)$$

其中, Tanh 为激活函数, 用于将 $H^{(l+1)}$ 限制为 $(-1, 1)$ 之间。得益于 GCN 对邻居节点的聚合能力以及激活函数 Tanh , 该步骤实现了通过单层 GCN 对节点重要性的有效学习, 该输出本质上表示每个节点的重要

性得分。GCN 结构参数如表 1 所示。

表 1 GCN 结构参数表
Table 1 GCN Structure Parameter

配置名称	具体信息	备注
输入维度	d	节点初始化维度
输出维度	1	经过单层 GCN 后节点特征维度
层数	2	实验验证层数为 2 时模型效果最佳

在获得图中各节点的注意力得分 S 之后, 通过对 S 进行排序, 得到节点的重要性指标 I , 见公式 (4)。

$$I = \text{Argsort}(S) \quad (4)$$

其中, Argsort 为排序函数。在获得节点排序得分 I 后, 首先确定在图中需要保留的节点数量, 见公式 (5)。

$$K = \max(1, \lfloor \rho n \rfloor) \quad (5)$$

其中, ρ 表示在图中需要保留的节点比例, K 为实际保留的节点数量。根据 I , K , 生成一个二值掩码矩阵 $\text{Mask} \in \mathbb{R}^{1 \times n}$, 指示被保留的节点, 矩阵中所有元素的和即为 K , 见公式 (6)。

$$\text{SUM}(\text{Mask}) = K \quad (6)$$

其中, Mask 用于指示哪些节点需要被保留, 哪些节点需要被舍弃。被选中保留的节点对应位置赋值为 1, 舍弃的节点对应位置赋值为 0, 从而在 Mask 向量中恰有 K 个位置为 1, $n-K$ 个位置为 0。随后, 依据生成的 Mask 对邻接矩阵 A 和节点特征矩阵 X 进行更新, 见公式 (7-8)。

$$A' = A \{[\text{Mask}, \text{Mask}]\} \quad (7)$$

$$X' = S[\text{Mask}] \cdot X[\text{Mask} :] \quad (8)$$

其中 $A' \in R^{k \times k}$, $X' \in R^{k \times d}$, 即为简化后的节点特征矩阵和邻接矩阵, 之后通过不同的池化策略对拓扑信息进行捕获, 并进行 concat 操作, 形成该层的全局特征表达, 见公式 (9)。

$$g_1 = \text{concat}(\text{avg}(X'), \text{max}(X')) \quad (9)$$

其中, $\text{avg}(X')$ 为平均池化用于捕捉图的整体分布特征, $\text{max}(X')$ 为最大池化用于突出图中最显著的特征。 g_1 表示经过第一层图简化后的图级全局特征向量。在后续的 GCN 层及其对应的简化步骤中, 利用当前层的邻接矩阵 A' 和节点特征 X' 作为输入, 并生成下一

次图的全局表示 g_2 ， g_2 表示经过第二层图简化后的图级全局特征向量。这两个特征向量加和后即为图全局拓扑编码器所生成的图的全局表征，见公式 (10)。

$$Z_1 = g_1 + g_2 \quad (10)$$

2.2.3 多尺度掩码卷积模块

该模块旨在通过两个大小不同的卷积核捕捉节点之间的多尺度关系。具体而言，大卷积核用于关注远距离节点之间的关系，而小卷积核则专注于邻近节点之间的关系。由于一个批次中图的大小各不相同，因此首先需要将各个批次中图的节点个数填充至该批次中最大节点数，为了避免因填充节点引入的噪声，该模型利用掩码算法，掩码算法如表 2 所示。

表 2 掩码算法

Table 2 Mask Algorithm

输入：特征张量列表：List of features, 填充值：padding_value

输出：填充后的特征：padded features, 掩码：mask

```

1 // Pad sequences to create uniform batch
2 padded_features= pad_sequence(features_tensors,
    padding_value=0)
3 // Generate mask for valid nodes
4 batch_size, max_node=
    shape(padded_features)[0:2]
5 mask=zeros(batch_size, max_nodes)
6 for i=1 to batch_size do
7     num_nodes ← shape(features_tensors[i])[0]
8     mask[i, 1:num_nodes] ← 1
9 end
10 return padded_features, mask

```

其中， $feature_tensors$ 表示一个批次图的张量列表， $padding_value$ 为填充数值设为 0， $padded_features \in R^{b \times m \times d}$ 表示填充后的特征张量，其中 b 为批次大小， m 为该批次图中最大节点数， d 为节点特征维度。对于节点数小于 m 的图，将其节点数填充至 m ，填充值为 $padding_value$ ，并生成对应掩码 $mask \in R^{b \times m}$ 。在该掩码中，填充位置设为 0，未填充位置设为 1。随后给定节点特征矩阵 X ，执行不同尺度的卷积，其中，小卷积核与大卷积核的大

小分别记作 S 和 L （其中 $S < L$ ）。该操作以并行方式进行不同卷积核尺度的卷积处理，其基于双分支卷积结构的输出定义见公式 (11)。

$$Q = BN(X * W^S) + BN(X * W^L) \quad (11)$$

其中 $*$ 代表卷积操作， $W^S \in R^{C_{in} \times C_{out} \times S}$ 和 $W^L \in R^{C_{in} \times C_{out} \times L}$ 分别表示小尺度卷积核与大尺度卷积，通过实验验证（见 4.3 超参数影响）将小尺度卷积核和大尺度卷积和分别设置为 3, 11，可捕获最优的长短距离依赖，例如：if($x > 0$) { ... use(x) ... } 语句中，以节点 x 为例；使用小卷积核大小为 3 的卷积层能捕获该句子的 if 条件以及对 x 的使用等局部依赖，而使用大卷积核为 11 的卷积层可以捕获句子之间的依赖，如对 x 的定义，对 x 的内存释放等信息，这种包含控制流信息的长距离依赖对漏洞检测尤为重要。因此卷积核的大小可以直接理解为捕获多大程度的程序依赖，采用不同大小卷积核即可捕获不同程度依赖。 C_{out} 和 C_{in} 分别表示卷积的输出和输入通道数。两个卷积分支的输出在经过批归一化 BN (Batch Normalization) 层后进行求和操作， Q 为经过 BN 后的卷积结果。随后进行掩码池化与特征池化，首先对掩码进行初步处理如公式 (12)。

$$mask = mask.unsqueeze(-1) \quad (12)$$

$unsqueeze$ 将 $mask \in R^{b \times m}$ 转化为 $mask \in R^{b \times 1 \times m}$ ，目的是方便与 K 做逐元素相乘，屏蔽无效位置，见公式 (13)。

$$Q = mask \cdot Q \quad (13)$$

这一步为逐元素乘法，把无效位置 ($mask = 0$) 的特征置为 0。保证后续池化不会被填充污染。之后分别对特征以及掩码进行平均池化，对特征做平均池化见公式 (14)。

$$Q_pooled = avg_pool1d(Q, 3) \quad (14)$$

此时特征中无效位置已经被置为 0。 avg_pool1d 为 PyTorch 的一维平均池化，池化窗口设为 3，对掩码做平均池化，统计池化窗口内有效元素的平均值。如果池化窗口内全是有效值，掩码池化结果接近 1，如果池化窗口内存在 0，结果是介于 (0,1) 的小数，见公式 (15)。

$$mask_pooled = avg_pool1d(mask, 3) \quad (15)$$

之后利用 $mask_pooled$ 对特征池化结果进行修正见公式(16)。

$$Q = \frac{Q_pooled}{mask_pooled.clamp.(1e-6)} \quad (16)$$

$clamp.(1e-6)$ 防止除 0 错误, 最终实现了带 $mask$ 的一维平均池化, 解决了序列中填充部分对平均池化结果的干扰。之后通过一个卷积层调整形状, 见公式 (17)。

$$Q = Dropout(Q * W^N) \quad (17)$$

其中 $W^N \in \mathbb{R}^{C_{out} \times C_{in} \times N}$, N 为卷积核, 随后进行 SUM 操作, 获取包含不同尺度节点关系的图特征, 见公式 (18)。

$$Z_2 = SUM(Q) \quad (18)$$

2.2.4 读出模块

在获取上述两部分的表征后, 使用一个门控机制融合两部分的特征并输出最终的检测结果, 见公式 (19)。

$$Z = MLP(\partial \odot Z_1 + (1-\partial) \odot Z_2) \quad (19)$$

其中, ∂ 是可训练参数, 初始值设为 (0.5), 维度为 1, $\odot$ 表示逐元素加权。在训练过程中, 门控权重 ∂ 会根据输入图的特征差异自动调整。如果全局拓扑信息更重要, ∂ 趋向 1; 如果局部卷积特征更关键, ∂ 趋向 0; 当两者互补时, ∂ 会在 (0,1) 区间自适应分配, 保证信息融合。这种自适应分配逻辑避免了单一超参数的限制。Z 是模型最终的检测结果。

在模型训练阶段, 首先从漏洞数据集中抽取源代码样本, 并构建相应的 CPG。每个节点均由预训练模型 (Word2Vec) 生成初始特征向量, 随后输入到 GTMC-VD 中进行表征学习。在训练过程中, 图全局拓扑编码器与多尺度掩码卷积模块分别提取多视角特征, 最后通过读出模块的门控机制进行融合, 并由 MLP 分类器输出预测结果。损失函数采用交叉熵损失, 对检测标签 (漏洞/非漏洞) 进行监督训练。在漏洞检测阶段, 输入的代码样本经过相同的数据预处理与特

征提取步骤, 得到的图表示输入到已训练好的 GTMC-VD 模型中, 最终输出对应的漏洞预测结果, 从而实现自动化漏洞检测。

3 实验设置

本节将介绍数据集信息、基准模型以及实验环境。

3.1 数据集

为了评估 GTMC-VD 模型的有效性, 本文在以下两个来自真实项目的数据集上进行了评估。

FFMPeg+Qemu^[31]数据集: 由四个大型开源 C 语言项目 (包括 Linux 内核、Qemu、Wireshark 和 FFmpeg) 中收集并人工标注而成, 包含 12460 个带有漏洞的函数和 14858 个无漏洞函数, 是一个相对平衡的数据集。

Reveal^[39]数据集: 由 Saikat 等人从 Chromium 和 Debian 项目的开发者或用户报告的漏洞中构建了一个真实世界的数据集, 该数据集中包含 2240 个有漏洞函数和 20494 个无漏洞函数, 更加贴近现实世界中漏洞代码的分布情况。

3.2 基准模型

VulDeePecker^[18]: 首先将源代码转换为代码片段 (Code Gadgets), 再利用 Word2Vec 生成向量表示, 随后使用 BiLSTM 学习代码的上下文特征。

Russell^[12]: 将源代码视为令牌序列, 使用 Word2Vec 生成向量表示, 使用 CNN 提取局部特征与模式。

SySeVR^[19]: 使用 BiLSTM 学习代码的上下文语义特征。相较于 VulDeePecker, SySeVR 更加关注代码语义的完整性以及对漏洞上下文的系统性分析。

Deign^[31]: 是一种基于 GNN 的自动化漏洞检测方法, 通过分析 CPG 来识别软件中潜在的安全漏洞。

Reveal^[39]: 以 GGNN 为核心模型, 将源代码转换为 CPG, 以便在图的节点和边之间传播信息并提取特征, 从而检测潜在的安全漏洞。

AMPLE^[10]: 提出一种图简化的代码表征, 以解决节点的远程依赖问题并利用 GNN 增强图特征学习。

DGVD^[40]: 提出 DGVD 模型, 结合双图神经网络与卷积特征增强, 有效提升了漏洞检测准确率, 达到 SOTA 效果。

3.3 实验环境

实验运行环境如表 3 所示。训练集、验证集和测试集采用随机划分策略,划分比例为 8:1:1;且本文采用的公共数据集由多个相互独立的项目构成,能够保证评估结果的可靠性。Batch size 设为 64,优化器选择为 RADAM,学习率为 0.0001,最大训练轮数为 100。

本文在评估中使用了分类任务中四种常用的性能评估指标:

$$Accuracy = \frac{TP + TN}{TP + TN + FP + FN} \quad (20)$$

Accuracy: 模型正确分类样本占总样本的比例。

$$Precision = \frac{TP}{TP + FP} \quad (21)$$

Precision: 预测为正的样本中正样本的数量。

$$Recall = \frac{TP}{TP + FN} \quad (22)$$

Recall: 表示模型识别出的正样本占实际正样本的比例。

$$F_1 \text{ Score} = \frac{Precision \times Recall}{Precision + Recall} \quad (23)$$

F_1 Score 是精确率与召回率的调和平均值; F_1 值越大,模型性能越稳健。

表 3 运行环境表

Table 3 Runtime environment configuration

配置名称	具体信息
CPU	Intel(R)Xeon(R)Gold 6133 CPU @ 2.50GHz
GPU	GPU NVIDIA RTX 3090 24G
RAM	64G
Hard Drive	512G M.2 SSD
Operating System	Ubuntu 18.04 desktop workstation
Python	3.9.21
pytorch	2.2.1
Dgl	2.0.0

4 实验结果

本章开展了相关实验,并对实验结果进行了分析。

4.1 对比试验

为了评估 GTMC-VD 的有效性,将其与六种具有代表性的漏洞检测方法进行了比较。

为保证实验的公平性,本文在相同实验条件下复现了 Devign, AMPLE 以及 Reveal 方法,并严格遵循相同的数据预处理、划分策略和超参数设置,对于其余缺乏完整可复现代码的基准模型(如 VulDeePecker、Russel 等),本文引用了 AMPLE 论文中公开的实验结果。由于这些结果基于相同的数据集和划分策略,因此具有可比性。对于基准模型 DGVD,由于该方法未公开源代码,无法在相同实验环境下复现,因此其结果可能与本文实验条件下的对比存在差异,且本文模型在召回率与 F_1 分数上明显优于该模型。具体的实验结果如表 4 所示。

从表 4 可以看出,GTMC-VD 在所有数据集和评估指标上均取得了具有竞争力的性能。具体而言,GTMC-VD 在所有数据集 Accuracy、Precision 和 F_1 Score 方面均优于所有基准模型。尽管其在第一个数据集上 Recall 低于 AMPLE,GTMC-VD 在其他数据集上的表现均明显超过 AMPLE。特别是,在第一个数据集上与基准模型 Devign 相比,GTMC-VD 在 Accuracy、Precision、Recall 和 F_1 Score 分别提高了 6.69%、4.43%、13.63%和 8.17%。虽然部分基线模型(如 AMPLE)在 Recall 指标上更高,但其 Precision 较低,导致 F_1 Score 弱于 GTMC-VD,这是由于 Recall 的提升往往以牺牲 Precision 为代价,而 GTMC-VD 在设计中更注重综合性能的平衡,从而在 Accuracy、Precision 和 F_1 Score 等关键指标上整体优于其他方法。此外,漏洞检测数据集存在样本分布不均衡的问题,GTMC-VD 在训练过程中更倾向于保证预测的可靠性,这也在一定程度上限制了 Recall 的提升。总体而言,GTMC-VD 在 Recall 指标上相对偏低是性能权衡的结果,其综合性能更为稳定和优越。

GTMC-VD 的优越性源于其关键的设计创新。GTMC-VD 使用 GCN 的输出作为节点重要性评分,并基于该评分有效简化了图结构,减少了图中节点数量,通过多级池化直接获取图的全局特征。同时,GTMC-VD 通过多尺度卷积来捕捉不同的节点依赖。此外,引入了掩码机制,以避免在处理变长图数据时由于填充节点带来的噪声干扰。

总体而言,这些改进使得 GTMC-VD 在漏洞检

测任务上超越了现有基准模型。在计算复杂性方面，GTMC-VD 的主要开销来自三个部分：（1）图全局拓扑编码器中的 GCN，其复杂度约为 $O(|E|d)$ ，其中 $|E|$ 为边数， d 为特征维度。该模块在特征提取过程中还引入了简化操作，能够显式减少图节点数量且计算简洁，从而在一定程度上降低整体复杂度；（2）多尺度掩码卷积模块，其复杂度随卷积核大小和节点数线性增长；（3）门控融合与 MLP 分类器，开销相对较小。因此，GTMC-VD 的整体复杂度与图的节点数和边数

近似呈线性关系，能够在中等规模的漏洞检测数据集上高效运行。此外，在参数量方面，GTMC-VD 仅包含 0.18M 参数，显著小于 Devign (0.31M) 和 AMPLE (0.92M)。这一轻量化特性不仅降低了模型的存储与计算成本，也进一步证明了 GTMC-VD 在复杂性上的优势。需要说明的是，本文实验主要基于公开漏洞数据集，规模有限，尚未在超大规模图数据上进行验证。

表 4 GTMC-VD 在不同数据集和指标上基线的对比表
Table 4 Comparison of baselines on different datasets and metrics

基准模型	FFMPeg+Qemu				Reveal			
	Accuracy	Precision	Recall	F1 Score	Accuracy	Precision	Recall	F1 Score
Vuldeepcker	49.61	46.05	32.55	38.14	76.37	21.13	13.10	16.17
Russell	57.60	54.76	40.72	46.71	68.51	16.21	52.68	24.79
Sysevr	47.85	46.06	58.81	51.66	74.33	40.07	24.94	30.74
Devign	56.52	52.57	61.05	56.49	89.01	29.46	35.10	32.03
Reveal	61.38	56.14	65.31	60.38	87.94	26.62	46.60	33.88
AMPLE	54.02	49.39	84.40	61.47	90.66	36.37	54.55	43.64
DGVD	64.23	61.45	59.33	60.37	、	、	、	、
GTMC-VD	63.21	57.00	74.68	64.66	91.21	41.82	48.94	45.10

4.2 消融实验

为了研究图全局拓扑编码器和多尺度掩码卷积模块对检测性能的贡献，本文设计了一系列消融实验。具体而言，评估了三种模型配置：仅使用图全局拓扑编码器模型、仅使用多尺度掩码卷积模型，以及同时使用两者进行表示学习的模型。相关实验结果如表 5 所示。从表 5 可以明显看出，单独使用任一部分所得到的 Accuracy、Precision、Recall 和 F_1 Score 均低于 GTMC-VD。这表明图全局拓扑编码器和多尺度掩码卷积模块对检测性能均有积极影响。

表 5 消融实验表

Table 5 Performance comparison of different model configurations

模型	Accuracy	Precision	Recall	F1 Score
----	----------	-----------	--------	----------

图全局拓				
扑编码器	60.13	54.64	67.85	60.53
多尺度掩				
码卷积	58.33	54.57	59.10	56.74
GTMC-VD	63.21	57.00	74.68	64.66

为探究这两部分的语义差异与互补性，本文对其嵌入表示的分布进行了可视化分析。如图 2 所示，采用 t-SNE 方法将两个空间中的高维嵌入映射到二维空间，横纵坐标为优化过程中生成的新维度，是对高维空间的映射。结果显示，两类嵌入展现出明显不同的聚类模式与较大程度的分离性，说明两个部分编码了不同分布结构的信息，且在高维空间中高度区分，语义差异明显。此外，本文还计算了同一样本在两种嵌

入之间的余弦相似度，并将其以热力图的形式展示在图 3 中。横纵坐标分别为指示样本，右侧纵坐标（0-0.7）指示样本之间的相似度，从图可以得出，样本大部分处于 0.4-0.6 之间表明两者提取到的特征存在一定相关性。综上，定量评估与定性分析结果共同证实，全局拓扑编码器和多尺度掩码分别捕捉了输入图中独立且互补的信息特征,其解耦但协同的特性丰富了模型的表达能力，显著提升了漏洞检测性能。

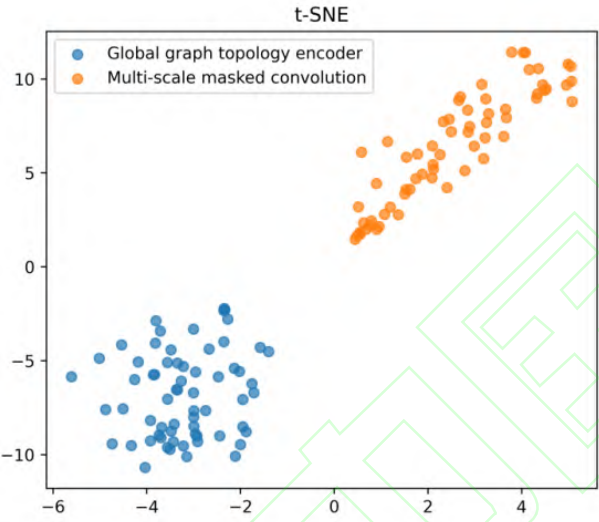


图 2 t-SNE 可视化
Fig.2 t-SNE visualization embedding

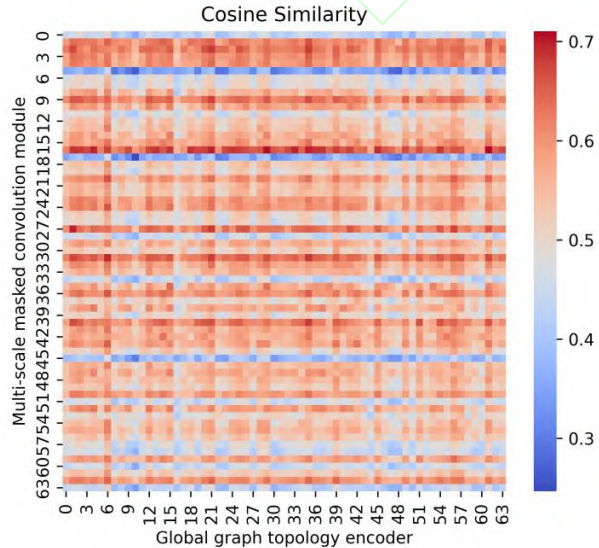


图 3 余弦相似度可视化
Fig.3 Cosine similarity visualization

为了研究读出模块中门控机制对模型性能的影响，设计了三种融合方式并对比其对应的模型性能，分别为固定加权 $Z = \text{MLP}(0.5Z_1 + 0.5Z_2)$ ，简单拼接 ($Z = \text{MLP}[Z_1 || Z_2]$)，以及本文的门控机制，相关实验结果如表 6 所示，从表 6 可得，基于门控机制的融合方式在四个指标上均优于前两种融合方式，这表明通过门控机制动态加权，模型取得了更优的效果。此外从图 2, 3 可知，两个特征相关性位于 0.5 附近，说明两个模块提取到的信息是互补的，侧面证明了门控的重要性。

表 6 特征融合方式对比表

Table 6 Comparison Table of Feature Fusion Methods				
融合方式	Accuracy	Precision	Recall	F1 Score
静态加权	59.63	54.77	72.89	62.55
简单拼接	60.19	55.36	71.80	62.52
GTMC-VD	63.21	57.00	74.68	64.66

4.3 超参数影响

GTMC-VD 的超参数包括图全局拓扑编码器中的简化层数和节点保留比例以及多尺度掩码卷积中卷积核的大小。

为验证这些超参数的影响，本文设计了相应实验，在 FFMpeg+Qemu^[31]数据集上进行了实验，实验结果如表 7 所示。

为了验证图简化层数的影响，本文进行了三组实验：分别将简化层数设置为单层、双层和三层。从值上看，单层和双层之间差异不大。然而，在单层简化的实验中，所有指标均处于较低水平。这是由于单层简化只能提取较为粗略的图结构特征，难以有效区分噪声与真实漏洞特征。通过实验结果进一步发现，当简化层数增加至 3 层时，模型性能反而下降，这是因为随着简化层数增加，图中节点较少，甚至在一些节点数较少的图中仅保留了一个节点，导致特征大量丢失，从而影响模型效果。

表 7 节点保留比例对 GTMC-VD 的影响

Table 7 The impact of node retention ratio on GTMC-VD					
模型	比例 (α)	Accuracy	Precision	Recall	F1 Score
单层	0.5	56.57	52.34	68.25	59.24
单层	0.8	60.12	54.63	67.84	60.53
双层	0.8,0.5	62.67	59.36	61.17	60.25

双层	0.7,0.7	59.72	53.91	73.16	62.08
三层	0.5	62.33	58.38	64.59	61.33
双层	0.5,0.8	63.21	57.00	74.68	64.66

为了研究节点保留比例的影响,本文固定图简化层数为 2,设计了两组对比实验:节点保留比例分别为 (0.8,0.5) 与 (0.7,0.7)。从表 7 可知,保留比例为 (0.8,0.5) 时模型性能较差。这是因为漏洞相关特征大多集中于程序的控制流与数据流中,涉及节点数较少。过多地保留节点会使大量与漏洞无关的特征被保留下来,进而影响模型判别能力。而当节点保留比例为 (0.5,0.8) 时,模型实现了“先粗后精”的逐层简化策略,检测性能得到明显提升。实验结果显示:在前期进行粗略简化、后期保留细粒度信息的方式,能最大限度保留关键拓扑结构,降低噪声干扰,提升模型表现。

图 4 展示了在不同大小的大卷积核与小卷积核设置下 GTMC-VD 模型的 F1 Score 表现。横轴为小卷积核,纵轴为大卷积核,当大卷积核与小卷积核的大小分别设置为 11 和 3 时,GTMC-VD 达到了最佳性能。若两种卷积核大小相同,则模型的 F_1 Score 明显降低,说明仅采用一种尺度的卷积核无法有效学习图表示。

此外,大卷积核的尺寸不能无限增大;当其大小达到 13 时,模型性能开始下降。综上所述,模型在利用大卷积核捕捉远距离节点关系的同时,通过小卷积核保留图结构中的局部信息,可以实现性能的优化。

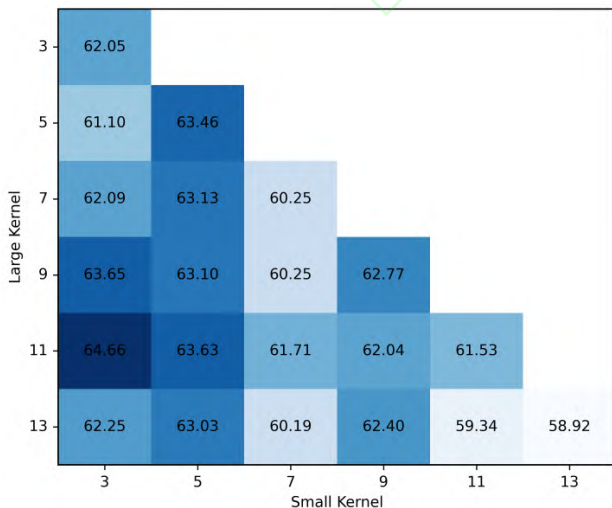


图 4 卷积核大小对 GTMC-VD 的 F1 得分的影响

Fig.4 Impact of kernel sizes on the F1 score of GTMC-VD

5 结论

本文针对现有图神经网络在软件漏洞检测任务中存在的局限性,设计并实现了 GTMC-VD。该模型首先将源代码转换为图表示,并从图全局表征与图中节点间不同尺度依赖两个维度进行联合建模。在图全局拓扑编码器中,利用 GCN 的输出对图结构进行简化,利用多层设计实现逐层优化,并利用不同池化方法获取不同尺度全图表征;在多尺度掩码卷积模块中,模型利用不同大小卷积核提取不同尺度信息,并通过掩码机制避免了在处理变长图数据时引入噪声。最终,模型采用门控机制动态融合两个通道的输出以生成最终预测结果。为验证所提出方法的有效性,在两个公开数据集上进行了大量实证实验。实验结果表明,GTMC-VD 在多个指标上均优于现有主流基准模型,展现出良好的性能。此外,实验结果进一步表明,图全局拓扑编码与多尺度掩码卷积所提取的特征在语义上具有互补性,显著增强了模型的整体表达能力,尽管两者功能上有所区别,但如图三所示两者所提取中的特征并非完全无关。如果忽略这种相关性,可能导致关键信息的丢失。未来的研究应探索一种简洁高效的方式用于融合两者的共享信息。

参考文献:

- [1] Synopsys. “2025 Open Source Security and Risk Analysis” Report [EB/OL].(2025).<https://www.blackduck.com>.
- [2] Lin G, Wen S, Han Q L, et al. Software Vulnerability Detection Using Deep Neural Networks: A Survey[J]. Proceedings of the IEEE, 2020, PP(99): 1-24. DOI:10.1109/JPROC.2020.2993293.
- [3] 苏小红,郑伟宁,蒋远,等.基于学习的源代码漏洞检测研究与进展[J].计算机学报, 2024, 47(2): 337-374. DOI:10.11897/SP.J.1016.2024.00337.Su X H, Zheng W N, Jiang Y, et al. Research and Progress on Learning-Based Source Code Vulnerability Detection, 2024, 47(2): 337- 374. DOI: 10.11897/SP.J.1016.2024.00337.
- [4] Zhou X, Cao S C, Sun X B ,et al.Large language model for vulnerability detection and repair:Literature review and the road ahead[J]. ACM Transactions on Software Engineering and Methodology, 2024,34: 1-31.
- [5] Shestov A, Levichev R, Mussabayev R ,et al.Finetuning large language models for vulnerability detection[J].IEEE Access,2025,13: 38889-38900.
- [6] Chen C, Su J Z, Chen J C ,et al.When ChatGPT meets smart contract vulnerability detection: How far are we?[J].

- ACM Transactions on Software Engineering and Methodology, 2025, 34(4):100.
- [7] Pang Y, Liu X, Huang T, et al. Graph-based Contract Sensing Framework for Smart Contract Vulnerability Detection[J]. IEEE Transactions on Big Data, PP[2025- 09-10]. DOI:10.1109/TBDATA.2025.3594303.
 - [8] Yamaguchi F, Golde N, Arp D, et al. Modeling and Discovering Vulnerabilities with Code Property Graphs[J]. IEEE, 2014. DOI:10.1109/SP.2014.44.
 - [9] Scarselli F, Gori M, Tsoi A C, et al. The Graph Neural Network Model[J]. IEEE Transactions on Neural Networks, 2009, 20(1): 61. DOI:10.1109/TNN.2008. 2005605.
 - [10] Wen X C, Chen Y, Gao C, et al. Vulnerability detection with graph simplification and enhanced graph representation learning[C]//Proceedings of the 45th International Conference on Software Engineering. Melbourne, Australia: IEEE, 2023: 2275-2286.
 - [11] Mikolov T, Chen K, Corrado G, et al. Efficient estimation of word representations in vector space[C]//Proceedings of the International Conference on Learning Representations. Scottsdale, USA: ICLR, 2013.
 - [12] Russell R L, Kim L, Hamilton L H, et al. Automated Vulnerability Detection in Source Code Using Deep Representation Learning[J]. IEEE, 2018. DOI:10.1109/ICMLA.2018.00120.
 - [13] Yan H, Luo S L, Pan L M, et al. HAN-BSVD: A hierarchical attention network for binary software vulnerability detection[J]. Computers & Security, 2021, 108: 102286.
 - [14] Hovsepyan A, Scandariato R, Joosen W, et al. Software Vulnerability Prediction using Text Analysis Techniques[C]//International Workshop on Security Measurements and Metrics. 2012. DOI:10.1145/2372225.2372230.
 - [15] Brown P F, Pietra V J D, Souza P V D, et al. Class-based n-gram models of natural language[J]. Computational Linguistics, 1992, 18(4):467-479. DOI:http://dx.doi.org/.
 - [16] Krizhevsky A, Sutskever I, Hinton G E. ImageNet classification with deep convolutional neural networks[J]. Communications of the ACM, 2017, 60(6): 84-90.
 - [17] Mikolov T, Zweig G. Context dependent recurrent neural network language model[C]//Spoken Language Technology Workshop. IEEE, 2013. DOI:10.1109/SLT.2012. 6424228.
 - [18] Li Z, Zou D, Xu S, et al. VulDeePecker: A Deep Learning-Based System for Vulnerability Detection[C]// Network and Distributed System Security Symposium. 2018. DOI:10.14722/ndss.2018.23165.
 - [19] Li Z, Zou D, Xu S, et al. SySeVR: A Framework for Using Deep Learning to Detect Software Vulnerabilities[J]. IEEE Transactions on Dependable and Secure Computing, 2021, PP(99):1-1. DOI:10.1109/TDSC.2021.3051525.
 - [20] Chung J, Gulcehre C, Cho K H, et al. Empirical Evaluation of Gated Recurrent Neural Networks on Sequence Modeling[J]. Eprint Arxiv, 2014. DOI:10. 48550/arXiv.1412.3555.
 - [21] Hochreiter S, Schmidhuber J. Long Short-Term Memory[J]. Neural Computation, 1997, 9(8): 1735-1780. DOI:10.1162/neco.1997.9.8.1735.
 - [22] Tian J, Xing W, Li Z. BVDetector: A Program Slice-based Binary Code Vulnerability Intelligent Detection System[J]. Information and Software Technology, 2020, 123:106289. DOI:10.1016/j.infsof. 2020.106289.
 - [23] Cho K, Van Merriënboer B, Gulcehre C, et al. Learning Phrase Representations using RNN Encoder-Decoder for Statistical Machine Translation[J]. Computer Science, 2014. DOI:10.3115/v1/D14-1179.
 - [24] Vaswani A, Shazeer N, Parmar N, et al. Attention is all you need[C]//Proceedings of the 31st International Conference on Neural Information Processing Systems. Red Hook, NY: Curran Associates Inc., 2017: 6000-6010.
 - [25] Tao W, Su X, Ke Y, et al. Transformer-based statement level vulnerability detection by cross-modal fine-grained features capture[J]. Knowledge-Based Systems[2025- 09-10].
 - [26] Hanif H, Maffei S. VulBERTa: Simplified source code pre-training for vulnerability detection[C]//2022 International Joint Conference on Neural Networks (IJCNN). Piscataway, NJ: IEEE, 2022: 1-8.
 - [27] Rahman M, Ceka I, Mao C, et al. Towards causal deep learning for vulnerability detection[C]//2024 IEEE/ACM 46th International Conference on Software Engineering (ICSE). New York: ACM, 2024: 1888-1898.
 - [28] 耿辰, 常舒予, 黄海平. 零样本场景下基于提示工程的智能合约漏洞检测研究[J]. 信息对抗技术, 2024, 3(2): 70-81. Geng C, Chang S Y, Huang H P. Prompt engineering for smart contract vulnerability detection in zero-shot scenarios[J]. Information Countermeasure Technology, 2024, 3(2):70-81.
 - [29] Du X Y, Zheng G, Wang K X, et al. Vul-RAG: Enhancing LLM-based vulnerability detection via knowledge-level RAG[J]. arXiv preprint arXiv:2406.11147, 2024.
 - [30] Siow J, Liu S Q, Xie X F, et al. Learning program semantics with code representations: An empirical study[C]//2022 IEEE International Conference on Software Analysis, Evolution and Reengineering (SANER). Piscataway, NJ: IEEE, 2022: 554-565.
 - [31] Zhou Y, Liu S Q, Siow J, et al. Devign: Effective vulnerability identification by learning comprehensive program semantics via graph neural networks[C]// Proceedings of the 33rd International Conference on Neural Information Processing Systems. Red Hook, NY: Curran Associates Inc., 2019: 10197-10207.

- [32] Li Y, Tarlow D, Brockschmidt M, et al. Gated Graph Sequence Neural Networks[J]. Computer Science, 2015. DOI:10.48550/arXiv.1511.05493.
- [33] Jiang B, Zhang Z, Lin D, et al. Semi-Supervised Learning With Graph Learning-Convolutional Networks[C]//2019 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR). IEEE, 2020. DOI:10.1109/CVPR.2019.01157.
- [34] Zheng W N, Jiang Y, Su X H. VuISPG: Vulnerability detection based on slice property graph representation learning[C]//2021 IEEE 32nd International Symposium on Software Reliability Engineering (ISSRE). Piscataway, NJ: IEEE, 2021: 457-467.
- [35] Schlichtkrull M, Kipf T N, Bloem P, et al. Modeling relational data with graph convolutional networks[C]// The Semantic Web: 15th International Conference, ESWC 2018. Cham: Springer, 2018: 593-607.
- [36] Li M, Li C F, Li S L, et al. ACGVD: Vulnerability detection based on comprehensive graph via graph neural network with attention[C]//Information and Communications Security: 23rd International Conference, ICICS 2021. Cham: Springer, 2021: 243-259.
- [37] Liu Z, Zhou J. Graph attention networks[M]// Introduction to graph neural networks. Singapore: Springer, 2020: 39-41.
- [38] Ghaffarian S M, Shahriari H R. Neural Software Vulnerability Analysis using Rich Intermediate Graph Representations of Programs[J]. Information Sciences, 2020. DOI:10.1016/j.ins.2020.11.053.
- [39] Chakraborty S, Krishna R, Ding Y, et al. Deep Learning based Vulnerability Detection: Are We There Yet[J]. IEEE Transactions on Software Engineering, 2021(01). DOI: 10.1109/TSE.2021.3087402.
- [40] Ling M, Tang M, Bian D, et al. A dual graph neural networks model using sequence embedding as graph nodes for vulnerability detection[J]. Information and Software Technology, 2025, 177(000). DOI:10.1016/j.infsof.2024.107581.



黄安博 (2001—), 男, 山西省临汾市, 硕士研究生, 主要研究方向为智能信息处理。HUANG Anbo, born in 2001, M.S. candidate. His research interest is image and intelligent information processing.



曲海成 (1981—), 男, 博士, 副教授, 硕士生导师, CCF 专业会员, 主要研究方向为遥感影像快速处理、智能大数据处理。QU Haicheng, born in 1981, Ph.D., associate professor, M.S. supervisor, member of CCF. His research interests include remote sensing image rapid processing and intelligent big data processing.



姜庆玲 (1982—), 女, 硕士, 副教授, CCF 专业会员, 主要研究方向为物联网应用技术, 物联网应用服务和物联网技术服务等。JIANG Qingling, born in 1982, M.S. member of CCF. Her research interests include IoT application technology, IoT application services, and IoT technical services.